

# Train. Observe. Understand the result.

A practical field guide to the first cuboid model. One training run, three reasons to stop, and evidence that tells us what to investigate next.

## 01 Train + measure

Prediction quality, group curves, failures.

## 02 Demonstrate

Animate the first model before comparisons.

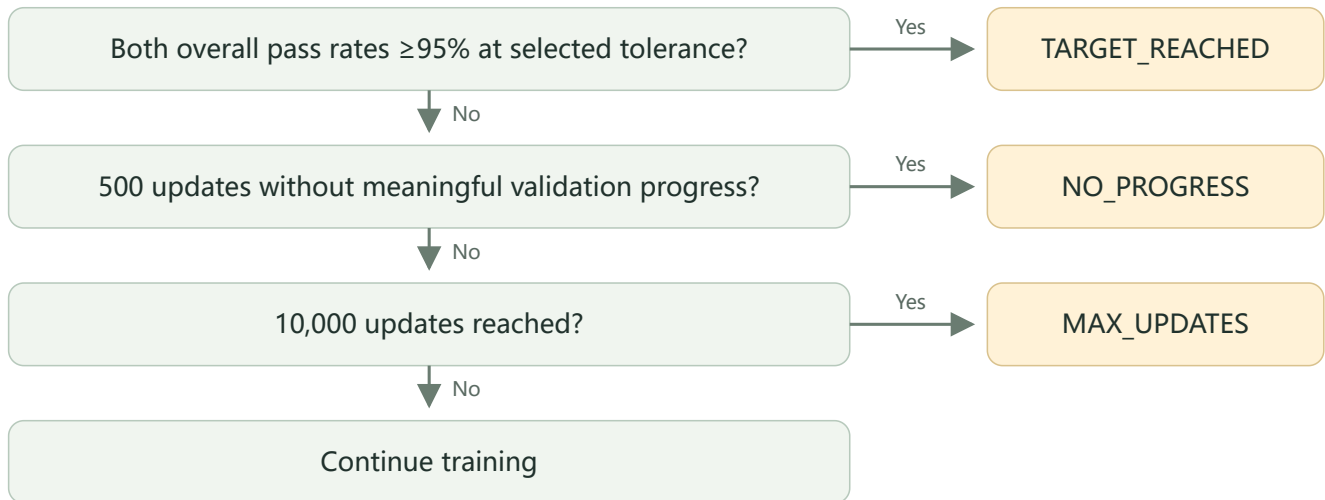
## 03 Compare

Change one main factor at a time.

**Two choices remain:** the stopping tolerance (coarse / intermediate / fine) and the numerical minimum improvement, `min_delta`. Confirmed: both overall pass rates  $\geq 95\%$ , patience 500 updates, cap 10,000 updates. No motion-group acceptance gate.

# The stop checker at a glance

Evaluate the current model on the full training and validation sets. All three rules live in one decision component; checkpoint saving is a separate responsibility.



Update the progress reference before checking patience. If patience and budget coincide, record both. Numerical failures abort separately. Never execute update 10,001.

## ▼ A ruler is not a coverage target

An example with displacement error **0.006 m** and velocity error **0.008 m/s** passes coarse and intermediate tolerances, but fails fine tolerance.

| Coarse • 0.1                     | Intermediate • 0.01              | Fine • 0.001                     |
|----------------------------------|----------------------------------|----------------------------------|
| <b>99%</b>                       | <b>96%</b>                       | <b>72%</b>                       |
| Illustrative validation coverage | Illustrative validation coverage | Illustrative validation coverage |

If intermediate is selected, this validation result meets the 95% target. The full training-set rate must also reach 95%. These are examples, not measured results.

# 1. Define the experiment

**Learn displacement and final velocity from initial conditions, then measure performance on unseen conditions.** Inputs are  $[v_0, F, t_1, t]$ ; outputs are  $[d, v]$ . Use 8,000 training rows and 1,000 validation rows; reserve the 1,000 test rows for the frozen final choice. Mass and friction stay fixed.

| Setting                      | First-run configuration                                                    |
|------------------------------|----------------------------------------------------------------------------|
| Network / device             | $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$ ; CPU; float32             |
| Optimizer                    | Adam; learning rate 0.001; weight decay 0                                  |
| Batch size / maximum updates | 128 / 10,000                                                               |
| Loss                         | MSE over standardized outputs and samples                                  |
| Stagnation patience          | 500 optimizer updates without meaningful validation-MSE improvement        |
| Coverage gate                | Overall training AND validation joint pass rates must both be at least 95% |
| Group requirements           | Diagnostic only; no minimum group pass rate                                |

Before the run, select the stopping tolerance, set an absolute `min_delta` in standardized validation-MSE units, and record a seed. No extra warm-up is proposed: start the progress clock at update 0; the first possible stagnation stop is update 500.

**Tolerance is the ruler; coverage is how many samples satisfy that ruler.**

| Evaluation ruler | Displacement error must be below | Velocity error must be below |
|------------------|----------------------------------|------------------------------|
| Coarse           | 0.1 m                            | 0.1 m/s                      |
| Intermediate     | 0.01 m                           | 0.01 m/s                     |
| Fine             | 0.001 m                          | 0.001 m/s                    |

At every evaluation, apply all three rulers to the same predictions. A sample passes only if BOTH errors are strictly below the chosen pair of tolerances. Select just ONE ruler for stopping; the other two reveal progress at different precision levels. These are not three training stages or three models, and reaching the coarse target does not claim fine accuracy.

## 2. Check that training works

**Do three small checks before spending the full training budget.**

| Check            | What to do                                                                                                                          | What it protects against                    |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Data and physics | Confirm columns, finite values, valid times and shapes; spot-check rest, sliding, stopping, reversal and breakaway labels.          | Learning from misread or incorrect targets. |
| Scaling          | Fit normalization on training data only; handle near-zero scales; verify inverse scaling restores the original values.              | Wrong units or information leakage.         |
| Learning         | Verify one update changes weights with finite loss/gradients; substantially reduce error on 32–64 fixed examples. Then start fresh. | Running a broken update loop.               |

The small fit is an implementation check, not evidence of generalization or a demand for perfect boundary fits.

## 3. Train and collect curves

**Each batch updates the model; scheduled evaluations measure the current model without changing it.**

Batch → predict → loss → clear gradients → backward → optimizer update → increment global step

Shuffle each epoch, keep the final partial batch, and continue using samples that already pass. Record batch loss every update. Evaluate at updates 0, 10, 20, 50, 100, then every 100 and at normal termination. At each check, evaluate the full training and validation sets using the SAME model in evaluation mode without gradients. This supplies comparable losses and the two overall pass rates needed by the stop checker; do not substitute the latest batch's pass rate.

Record update, epoch, sample exposures, elapsed time, split, MSE, and all three pass rates in CSV. Plot validation MSE and pass rate against updates, with pass rate fixed to 0–100%; also compare matched training and validation curves. Keep raw observations. A logarithmic shape or a plateau is not required.

## 4. Measure what is difficult

**Overall results decide target achievement; motion groups explain where the model struggles.**

Attach reference-derived `motion_group` metadata to each row when loading data, keeping row IDs and labels together through shuffling. Keep it separate from the four network inputs and two regression targets. Existing CSVs need not be overwritten. For the current positive-duration, fixed-physics task:

| Group                 | Reference definition                                                                |
|-----------------------|-------------------------------------------------------------------------------------|
| Always resting        | $v_0 = 0$ and $\text{abs}(F) \leq \mu_s * m * g$ ; no movement during the interval. |
| Moved then stopped    | Not always resting, and reference final velocity is zero.                           |
| Moving at observation | Reference final velocity is nonzero, including cases that reversed direction.       |

Use the reference's exact rest/stop logic, not a model-error tolerance, to label these synthetic data. The last group does NOT claim uninterrupted motion: reversal may include an instant of zero velocity. Check that groups are disjoint, exhaustive, and consistent with the reference; net displacement alone cannot establish whether an object ever moved.

Shuffling does not prevent group metrics. Retain each sample's two squared errors before averaging; use its group ID to accumulate sums and counts. The ordinary scalar batch mean still drives training. Use full-validation group curves as the main diagnostic, rather than noisy mixed-batch averages. Aggregate by sample count, not by averaging batch means; an empty group has count zero and undefined metrics.

At each check, record validation group counts, MSE, joint pass rates for all rulers, and displacement/velocity MAE, P95 and maximum error. Also report overall physical errors. Track the overlapping breakaway slice  $v_0 = 0$  and  $\text{abs}(\text{abs}(F) - 3.924 \text{ N}) \leq 0.2 \text{ N}$ , without a separate acceptance gate.

At update 0, score two simple baselines: always predict  $[0, 0]$ , or predict constant velocity  $[v_0 * t, v_0]$ . They reveal whether the network improves on trivial guesses. Keep per-row errors at updates 0 and 10, if reached, to inspect early improvement and regression.

## 5. Stop checker: three reasons to finish

**One decision component checks success, lack of progress, and the maximum budget. Stopping is not always success.**

Inputs are the global update count, matched overall training/validation pass rates for the SELECTED ruler, and validation MSE. Persistent state holds the progress-reference MSE and its update, plus a separate absolute best MSE and checkpoint. The stop checker makes decisions; checkpoint saving preserves versions.

| Decision               | Exact condition                                                                                     | Outcome and model selection                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Target reached         | Both overall pass rates are at least 95%. No group gate.                                            | <code>TARGET_REACHED</code> : save and select this first-passing checkpoint. Applies at update 0 too.               |
| No meaningful progress | At a check, at least 500 updates have elapsed since the last meaningful validation-MSE improvement. | <code>NO_PROGRESS</code> : stop for diagnosis; select the lowest-validation-MSE checkpoint.                         |
| Budget exhausted       | Update count reaches 10,000, even if improvement continues.                                         | <code>MAX_UPDATES</code> : evaluate and stop; select the lowest-validation-MSE checkpoint unless the target is met. |

**Decision order at a scheduled check:** verify finite metrics → save any new absolute best → check target achievement → update the progress reference → check 500-update patience → check 10,000-update cap. If patience and the cap coincide, record both flags and use `NO_PROGRESS` as the primary reason. Never execute update 10,001. Numerical failures abort separately and cannot count as success.

Initialize the progress reference at update 0. A meaningful improvement is `current_val_mse < progress_reference_mse - min_delta`. Only then replace the reference and reset its update counter. Smaller improvements may accumulate against that unchanged reference, while every new absolute minimum still saves a best checkpoint. For example, an improvement at update 200 followed by none for 500 updates permits stopping at the check at update 700. Actual stopping occurs at checks, so the wait may be slightly longer than 500 updates. The numerical `min_delta` remains to be chosen.

## 6. Save evidence and diagnose the result

Finish with a recoverable model, readable curves, and an explanation of its limits.

| Artifact        | Keep                                                                                                                                                                              | Purpose                                              |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Run record      | Config, data hashes/row IDs, seed, software/device, normalization, updates/exposures/time, termination reason.                                                                    | Reproduce and interpret the result.                  |
| Checkpoints     | At every scheduled evaluation, plus final and selected model; save weights, model config and normalization. Save optimizer/RNG state for continuation. Verify reload predictions. | Recover a model and later animate training progress. |
| Learning curves | Matched train/validation metrics and full-validation group curves with counts.                                                                                                    | See generalization and harder motion groups.         |
| Failure table   | Selected and last valid checkpoint: row ID, input, reference group, target, prediction, physical errors and severity.                                                             | Understand how far failures miss the target.         |
| Result summary  | Selected ruler/coverage, measured pass rates, group results, physical error summaries, baselines, stop reason and limitations.                                                    | Explain what was actually achieved.                  |

For failures at the selected ruler, use `severity = max(abs(d_error)/tau_d, abs(v_error)/tau_v)` ; retain physical units too. Summarize failed-only errors and counts; inspect representative and worst cases. If none fail, report zero failures and leave failed-only statistics undefined.

**Diagnosis sequence:** inspect the raw trend and stop settings → compare matched train/validation results → locate failing groups and examples → propose one explanation and one controlled follow-up. Do not automatically enlarge the model or extend training. Timing is supporting context, not a primary claim.

Use validation for this feedback loop. Freeze the training protocol and model selection before the final test evaluation; report test results separately and do not tune against them. A training target being reached is distinct from final test acceptance. State that this is synthetic one-dimensional reference agreement, not general 3D or real-world validation.

## 7. Demonstrate, then compare

**The first-model animation is the second project milestone, before other experiments.** Use saved training checkpoints and the analytical reference; Models B/C are not prerequisites. Defer animation design until after the first training result is reviewed.

After the demonstration, compare architectures, data sizes and losses one factor at a time. Screen 4 → 16 → 2, 4 → 32 → 2, 4 → 16 → 16 → 2 and 4 → 32 → 32 → 2 with matched data and protocols; repeat promising comparisons with three seeds. Use fresh models and subset-specific normalization for nested 500/1,000/2,000/4,000/8,000-row comparisons. Study rollouts separately with matched force schedules, explicit position tracking and out-of-range checks. See the notes for comparison limitations.

English working document · [Execution plan](#) · [Supporting notes](#)

This guide describes planned work, not a completed training run.